TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS • BACK-END

Persistindo dados: JSON e PostgreSQL

O CRUD da aula passada, agora com dados que sobrevivem ao reinício do servidor

```
const { rows } = await pool.query(
  'SELECT * FROM alunos'
);
```

PROFESSOR

Tarik Ponciano

BACK-END COM NODE.JS

Roteiro

01 **Por que persistir**
Memória, arquivo ou banco: onde os dados moram

02 **Arquivo JSON**
Módulo fs, async/await e o CRUD gravando num arquivo

03 **PostgreSQL**
Biblioteca pg, consultas com parâmetros e o CRUD em SQL

04 **Fechamento**
As três versões lado a lado, desafio e erros comuns

NESTE MATERIAL

Exemplo explicado
O professor resolve um caso: a API de alunos.

Prática
Você aplica em outro caso: a API da lanchonete.

Código JavaScript
Com import, pronto para o server.js.

ONDE ESTAMOS

Da memória para o banco

AULA PASSADA • MEMÓRIA

Lista no servidor

```
let alunos = [
  { id: 1, nome: 'Ana' }
];
```

Rápido de fazer, mas **some** quando o servidor reinicia.

HOJE • PARTE 1

Arquivo JSON

```
await writeFile(
  'alunos.json', texto
);
```

Os dados ficam num **arquivo** dentro do projeto.

HOJE • PARTE 2

PostgreSQL

```
await pool.query(
  'SELECT * FROM alunos'
);
```

Os dados ficam no **banco** que você já conhece do pgAdmin.

As rotas, os métodos e o JSON de entrada e saída **continuam os mesmos**. Muda só onde os dados moram.

SEÇÃO 01

Por que persistir

01

Memória, arquivo ou banco: onde os dados moram — e por que isso importa.

01 • Persistência
02 • Arquivo JSON
03 • PostgreSQL
04 • Fechamento

01 • PERSISTÊNCIA

Reiniciou, perdeu

UMA TARDE NA API DE ALUNOS

1. `POST /alunos`
   {"nome": "Carla", "turma": "TDS-01"}
   201 • id 3

2. `Ctrl + C · node server.js`
   o servidor reinicia (ou o --watch reinicia ao salvar)

3. `GET /alunos/3`
   {"erro": "Aluno não encontrado"}
   404

A lista **let alunos = [...]** mora na memória do processo. Quando o processo termina, a memória é liberada — e a Carla vai junto.

> **Persistir**
> Guardar os dados **fora do processo**, num lugar que continua lá depois do reinício: um arquivo ou um banco de dados.

01 • PERSISTÊNCIA

Três lugares para os dados

	Memória	Arquivo JSON	PostgreSQL
Onde ficam	Na memória do processo	Num arquivo do projeto	No servidor de banco
Reiniciou o servidor	Perde tudo	Continua lá	Continua lá
Como buscar	find e filter	Lê o arquivo inteiro e filtra	SQL: WHERE, ORDER BY
Bom para	Testes e protótipos	Projetos pequenos	Sistemas com muitos usuários

Hoje você vai fazer o **mesmo CRUD** duas vezes: primeiro gravando num arquivo, depois no PostgreSQL.

SEÇÃO 02

Arquivo JSON

02

Gravar a lista num arquivo do projeto, com o módulo fs do próprio Node.

02 · ARQUIVO JSON

O módulo fs: o Node mexe em arquivos

```
import { readFile, writeFile } from 'node:fs/promises';

// lê o arquivo inteiro como texto
const texto = await readFile('./alunos.json', 'utf-8');

// grava o texto no arquivo (substitui o que tinha)
await writeFile('./alunos.json', texto);
```

node:fs/promises

Já vem com o Node: nada de npm install. O **node:** indica um módulo do próprio Node.

readFile

Devolve o conteúdo do arquivo. Com **'utf-8'**, ele vem como texto.

writeFile

Troca **todo** o conteúdo do arquivo pelo texto novo.

```
api-alunos/
├── alunos.json        novo
├── node_modules/
├── package.json
└── server.js
```

02 · ARQUIVO JSON

Texto e objeto: JSON.parse e JSON.stringify

LER

TEXTO · NO ARQUIVO

```
[{"id":1,"nome":"Ana"}]
```

JSON.parse(texto) →

OBJETO · NO JAVASCRIPT

```
[{ id: 1, nome: 'Ana' }]
```

GRAVAR

OBJETO · NO JAVASCRIPT

```
[{ id: 1, nome: 'Ana' }]
```

JSON.stringify(alunos) →

TEXTO · NO ARQUIVO

```
[{"id":1,"nome":"Ana"}]
```

Por que converter?

O arquivo só guarda texto. Para usar find, push e filter, o texto precisa virar objeto — e voltar a ser texto para gravar.

E o null, 2?

O **2** organiza o arquivo com 2 espaços de recuo. Fica legível para você abrir e conferir.

02 • ARQUIVO JSON

Por que async e await?

- Ler e gravar arquivo **leva tempo**. O Node não fica parado esperando.
- **await** espera o resultado antes de seguir para a próxima linha.
- Função que usa await precisa ser **async**.

Você já usou: o Inquirer também espera a resposta do teclado.

```
const nome = await input({ ... });
```

```
app.get('/alunos', async (req, res) ⇒ {
                   1
  const alunos = await lerAlunos();
                      2
  res.json(alunos);
});
```

1 • async
Avisa que a função vai esperar alguma coisa.

2 • await
Espera a leitura do arquivo terminar.

⚠ Esqueceu o await? A rota recebe uma **Promise**, não a lista — e responde {}.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Duas funções: ler e salvar

SERVER.JS · ANTES DAS ROTAS

```
import { readFile, writeFile } from 'node:fs/promises';

const ARQUIVO = './alunos.json';

async function lerAlunos() {
  const texto = await readFile(ARQUIVO, 'utf-8');
  return JSON.parse(texto);
}

async function salvarAlunos(alunos) {
  const texto = JSON.stringify(alunos, null, 2);
  await writeFile(ARQUIVO, texto);
}
```

alunos.json

```
[
  {
    "id": 1,
    "nome": "Ana",
    "turma": "TDS-01"
  }
]
```

Assim fica o arquivo gravado com stringify(alunos, null, 2).

⚠ Comece o arquivo com **[]**. Vazio, o JSON.parse dá erro.

02 • ARQUIVO JSON • NA NUVEM

Caminho certo e arquivo garantido

SERVER.JS • NO LUGAR DO const ARQUIVO

```
import { readFile, writeFile } from 'node:fs/promises';
import { existsSync } from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

// a pasta do server.js, em qualquer computador
const PASTA = path.dirname(fileURLToPath(import.meta.url));
const ARQUIVO = path.join(PASTA, 'alunos.json');

// primeiro deploy? cria o arquivo com uma lista vazia
if (!existsSync(ARQUIVO)) {
  await writeFile(ARQUIVO, '[]');
}
```

O problema

'./alunos.json' é procurado na pasta de onde o node foi iniciado, não na do server.js. Na nuvem, ele pode iniciar de outra pasta: ENOENT.

`import.meta.url`
O endereço do próprio server.js.

`fileURLToPath + dirname`
Viram a pasta onde ele está.

`path.join`
Junta pasta e nome com a barra certa, no Windows ou no Linux.

`existsSync`
Confere se o arquivo existe antes do primeiro uso.

Node 20.11 ou mais novo: **import.meta.dirname** faz o papel da PASTA em uma linha.

⚠ Em muitas nuvens, como o plano gratuito do Render, o disco é apagado a cada deploy ou reinício. O arquivo serve para testes; para guardar de verdade, use um banco — a parte 2 da aula.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Ler: GET com o arquivo

```
app.get('/alunos', async (req, res) => {
  const alunos = await lerAlunos();
  res.json(alunos);
});
```

```
app.get('/alunos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const alunos = await lerAlunos();
  const aluno = alunos.find(a => a.id === id);
  if (!aluno) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  res.json(aluno);
});
```

O QUE MUDOU

async na função da rota
Porque ela vai esperar a leitura.

await lerAlunos()
A lista vem do arquivo, a cada pedido.

O resto
Igual ao da aula passada.

TESTE

Abra o alunos.json, troque o nome da Ana à mão, salve e faça **GET /alunos/1** de novo.

PRÁTICA 1 • 20 MIN • INDIVIDUAL

O cardápio no arquivo

a. Na **api-lanchonete**, crie o **produtos.json** com os três produtos e apague a lista **let produtos**.

b. Escreva **lerProdutos()** e **salvarProdutos(produtos)**, com o caminho montado por **path.join**.

c. Reescreva **GET /produtos** e **GET /produtos/:id** lendo do arquivo.

d. Mude o preço da coxinha direto no arquivo e faça **GET /produtos/1**.

COMEÇO DO produtos.json

```
[
  {
    "id": 1,
    "nome": "Coxinha",
    "preco": 6.5,
    "categoria": "salgado"
  },
  ...
]
```

Depois: Pastel (7, salgado) e Refrigerante (5, bebida).

PENSE E RESPONDA Por que o GET viu a mudança no arquivo sem reiniciar o servidor?

02 · ARQUIVO JSON

Toda rota que altera: ler, mudar, salvar

1

Ler

`await lerAlunos()`

Traz a lista do arquivo.

→

2

Mudar

`push · find · filter`

Igual à aula passada.

→

3

Salvar

`await salvarAlunos()`

Grava a lista inteira de volta.

→

4

Responder

`res.status(...)`

Só depois de salvar.

Esqueceu de salvar?

A resposta sai certa, mas o arquivo não muda: no próximo GET, a alteração sumiu.

Por que salvar antes de responder?

Se a gravação falhar, o cliente não recebe um 201 de algo que não foi guardado.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Criar: POST no arquivo

```
app.post('/alunos', async (req, res) => {
  const { nome, turma } = req.body;
  if (!nome || !turma) {
    return res.status(400)
      .json({ erro: 'Informe nome e turma' });
  }

  const alunos = await lerAlunos();  // 1. ler
  const novo = { id: gerarId(alunos), nome, turma };
  alunos.push(novo);  // 2. mudar
  await salvarAlunos(alunos);  // 3. salvar

  res.status(201).json(novo);  // 4. responder
});
```

E O ID?

```
function gerarId(lista) {
  if (lista.length === 0) return 1;
  const ids = lista.map(a => a.id);
  return Math.max(...ids) + 1;
}
```

Por que não o proximoId?

Ele vivia na memória e voltava ao início a cada reinício. Agora o id novo é o maior id gravado + 1.

Teste

POST /alunos → 201. Abra o alunos.json: o aluno novo está lá.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Atualizar e remover no arquivo

PUT · ATUALIZAR

```
app.put('/alunos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const alunos = await lerAlunos();
  const aluno = alunos.find(a => a.id === id);
  if (!aluno) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  aluno.nome = req.body.nome ?? aluno.nome;
  aluno.turma = req.body.turma ?? aluno.turma;
  await salvarAlunos(alunos);
  res.json(aluno);
});
```

O find devolve o objeto da lista lida: mudar aluno.nome muda a lista, e o salvar grava tudo.

DELETE · REMOVER

```
app.delete('/alunos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const alunos = await lerAlunos();
  const resto = alunos.filter(a => a.id !== id);
  if (resto.length === alunos.length) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  await salvarAlunos(resto);
  res.status(204).send();
});
```

O filter não tirou ninguém? Então o id não existia: 404.

PRÁTICA 2 • 25 MIN • INDIVIDUAL

O cardápio que não esquece

a. **POST /produtos**: ler, criar com gerarId, salvar e responder 201.

b. **PUT /produtos/:id**: ler, trocar só o que veio no corpo, salvar.

c. **DELETE /produtos/:id**: ler, filtrar, salvar e responder 204 (ou 404).

Modelo: POST, PUT e DELETE de alunos com arquivo.

TESTES NO THUNDER CLIENT

`POST /produtos` — **201**

```
{ "nome": "Suco de cajá",
  "preco": 5, "categoria": "bebida" }
```

`PUT /produtos/1 · {"preco": 7}` — **200**

`DELETE /produtos/2` — **204**

Reinicie o servidor e faça GET /produtos:
o suco continua lá; o pastel, não.

PENSE E RESPONDA Abra o produtos.json depois de cada teste. O que mudou no arquivo?

SEÇÃO 03

PostgreSQL

03

O mesmo CRUD, agora com o banco que você já conhece do pgAdmin.

03 · POSTGRESQL

Você já sabe SQL

Thunder Client
pede GET /alunos

→

Express
a rota roda

→

Biblioteca pg
leva o SQL ao banco

→

PostgreSQL
executa e devolve as linhas

Rota	Comando	Exemplo em SQL
GET /alunos	SELECT	`SELECT * FROM alunos;`
GET /alunos/:id	SELECT	`SELECT * FROM alunos WHERE id = 3;`
POST /alunos	INSERT	`INSERT INTO alunos (nome, turma) VALUES ('Ana', 'TDS-01');`
PUT /alunos/:id	UPDATE	`UPDATE alunos SET turma = 'TDS-02' WHERE id = 3;`
DELETE /alunos/:id	DELETE	`DELETE FROM alunos WHERE id = 3;`

Preparando o banco no pgAdmin

QUERY TOOL • BANCO api_escola

```
CREATE TABLE alunos (
  id    SERIAL PRIMARY KEY,
  nome  VARCHAR(100) NOT NULL,
  turma VARCHAR(20)  NOT NULL
);

INSERT INTO alunos (nome, turma) VALUES
  ('Ana', 'TDS-01'),
  ('Bruno', 'TDS-01');
```

1. No pgAdmin, crie o banco **api_escola**.
2. Abra a Query Tool nesse banco e rode o código (ou crie a tabela pela interface).
3. Confira: **SELECT * FROM alunos;**

SERIAL: o banco gera o id

1, 2, 3… sozinho. Substitui o gerarId — por isso o INSERT não informa o id.

03 · POSTGRESQL · CONFIGURAÇÃO

Conectando com a biblioteca pg

DB.JS · ARQUIVO NOVO

```
import pg from 'pg';
const { Pool } = pg;

export const pool = new Pool({
  host: 'localhost',
  port: 5432,
  database: 'api_escola',
  user: 'postgres',
  password: 'sua_senha'
});
```

⚠ Senha no código só em aula. Num projeto real, ela vai para variáveis de ambiente e nunca sobe para o GitHub.

1 · INSTALE

```
npm install pg
```

2 · CRIE O DB.JS

Com os dados de conexão do seu PostgreSQL.

3 · IMPORTE NO SERVER.JS

```
import { pool } from './db.js';
```

Pool

Um conjunto de conexões prontas: cada consulta pega uma, usa e devolve.

Usando a Aiven?

Copie host, porta, banco, usuário e senha do painel e acrescente **ssl: { rejectUnauthorized: false }**.

03 · POSTGRESQL · ANATOMIA

Anatomia de uma consulta

```
const { rows } = await pool.query(
        (1)                (2)
  'SELECT * FROM alunos WHERE id = $1',
                        (3)
  [id]
   (4)
);
```

1 • O resultado
rows: as linhas, cada uma como objeto. Também vem rowCount.

2 • A consulta
Manda o SQL ao banco e espera a resposta: por isso o await.

3 • O SQL
O mesmo da Query Tool, com $1 no lugar do valor.

4 • Os valores
Na ordem: o primeiro vai no $1, o segundo no $2…

O QUE VOLTA `rows = [ { id: 3, nome: 'Carla', turma: 'TDS-01' } ]`

03 • POSTGRESQL • SEGURANÇA

Valores sempre como parâmetros

EVITE • JUNTAR TEXTO

```
pool.query(
  'SELECT * FROM alunos WHERE id = '
  + req.params.id
);
```

Se alguém pedir **/alunos/1 OR 1=1**, o banco recebe outro comando e devolve todos os alunos. Isso é **injeção de SQL**.

PREFIRA • PARÂMETROS

```
pool.query(
  'SELECT * FROM alunos WHERE id = $1',
  [req.params.id]
);
```

O pg envia o valor **separado** do comando. O banco trata como dado, nunca como SQL.

Vários valores? **$1, $2, $3** no SQL, na mesma ordem do array: '... VALUES ($1, $2)', [nome, turma]

Ler: GET com PostgreSQL

```
app.get('/alunos', async (req, res) => {
  const { rows } = await pool.query(
    'SELECT * FROM alunos ORDER BY id');
  res.json(rows);
});
```

```
app.get('/alunos/:id', async (req, res) => {
  const { rows } = await pool.query(
    'SELECT * FROM alunos WHERE id = $1',
    [req.params.id]);
  if (rows.length === 0) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  res.json(rows[0]);
});
```

O QUE MUDOU

lerAlunos() → pool.query
O banco busca; nada de ler arquivo.

find → WHERE id = $1
O filtro sai do JavaScript e vai para o SQL.

rows[0]
A consulta sempre devolve uma lista, mesmo com uma linha só.

TESTE

Insira um aluno pelo pgAdmin e faça **GET /alunos**: ele aparece na API.

PRÁTICA 3 • 25 MIN • INDIVIDUAL

O cardápio no PostgreSQL

a. No pgAdmin: banco **api_lanchonete**, a tabela ao lado e os três produtos com INSERT.

b. **npm install pg** e um **db.js** apontando para o api_lanchonete.

c. Reescreva **GET /produtos** e **GET /produtos/:id** com pool.query.

d. Teste: /produtos/2 dá **200**; /produtos/99 dá **404**.

A TABELA

```
CREATE TABLE produtos (
  id        SERIAL PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  preco     NUMERIC(6,2) NOT NULL,
  categoria VARCHAR(30)
);
```

Produtos: Coxinha (6.50, salgado), Pastel (7.00, salgado), Refrigerante (5.00, bebida).

PENSE E RESPONDA O preço voltou como "6.50", entre aspas. Por que o pg devolve NUMERIC como texto?

03 · POSTGRESQL · EXEMPLO EXPLICADO

Criar: INSERT com RETURNING

```
app.post('/alunos', async (req, res) ⇒ {
  const { nome, turma } = req.body;
  if (!nome || !turma) {
    return res.status(400)
      .json({ erro: 'Informe nome e turma' });
  }

  const { rows } = await pool.query(
    `INSERT INTO alunos (nome, turma)
     VALUES ($1, $2) RETURNING *`,
    [nome, turma]
  );
  res.status(201).json(rows[0]);
});
```

RETURNING *

O banco devolve a linha que acabou de criar, com o id que o SERIAL gerou.

Crase: SQL em várias linhas

O texto entre `` pode quebrar linha. Fica mais fácil de ler.

Sem ler, mudar e salvar

O INSERT já grava. O banco faz as três etapas sozinho.

Teste

POST /alunos → 201 com o id novo. Confira no pgAdmin.

03 · POSTGRESQL · EXEMPLO EXPLICADO

Atualizar e remover com SQL

PUT · ATUALIZAR

```
app.put('/alunos/:id', async (req, res) => {
  const { nome, turma } = req.body;
  const { rows } = await pool.query(
    `UPDATE alunos
     SET nome = COALESCE($1, nome),
         turma = COALESCE($2, turma)
     WHERE id = $3 RETURNING *`,
    [nome, turma, req.params.id]);
  if (rows.length === 0) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  res.json(rows[0]);
});
```

DELETE · REMOVER

```
app.delete('/alunos/:id', async (req, res) => {
  const { rowCount } = await pool.query(
    'DELETE FROM alunos WHERE id = $1',
    [req.params.id]);
  if (rowCount === 0) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  res.status(204).send();
});
```

COALESCE($1, nome): usa o valor novo; se ele não veio, mantém o atual. É o ?? do SQL.

rowCount: quantas linhas o DELETE apagou. Zero? O id não existia: 404.

Quando o banco falha: try e catch

```
app.get('/alunos', async (req, res) => {
  try {
    const { rows } = await pool.query(
      'SELECT * FROM alunos ORDER BY id');
    res.json(rows);
  } catch (erro) {
    console.error(erro.message);
    res.status(500).json({ erro: 'Erro no banco de dados' });
  }
});
```

try

Tenta rodar o bloco normalmente.

catch

Se algo der erro lá dentro, cai aqui. O motivo vai para o terminal; o cliente recebe um 500 em JSON.

Quando acontece

Banco desligado, senha errada, nome de tabela errado.

Teste: troque alunos por aluno no SQL e faça GET /alunos. O terminal mostra relation "aluno" does not exist.

PRÁTICA 4 · 25 MIN · INDIVIDUAL

CRUD completo no banco

a. **POST /produtos** com INSERT … RETURNING *.

b. **PUT /produtos/:id** com UPDATE e COALESCE.

c. **DELETE /produtos/:id** usando o rowCount.

d. **try/catch** nas rotas: erro no banco vira 500.

TESTES NO THUNDER CLIENT

`POST /produtos` — **201**

```
{ "nome": "Suco de cajá",
  "preco": 5, "categoria": "bebida" }
```

`PUT /produtos/1 · {"preco": 7}` — **200**

`DELETE /produtos/2` — **204**

`GET /produtos/2` — **404**

Depois, no pgAdmin: **SELECT * FROM produtos;**

PENSE E RESPONDA — Mude um preço pelo pgAdmin e faça GET pelo Thunder. Quem guarda a verdade dos dados agora?

SEÇÃO 04

Fechamento

04

As três versões lado a lado, o desafio em dupla e os erros mais comuns.

04 · FECHAMENTO

Mesma rota, três jeitos

Dentro de **GET /alunos/:id**, só muda a parte que busca o aluno:

MEMÓRIA

```
const aluno = alunos
  .find(a => a.id === id);
```

Some ao reiniciar o servidor.

ARQUIVO JSON

```
const alunos =
  await lerAlunos();
const aluno = alunos
  .find(a => a.id === id);
```

Sobrevive. Bom para projetos pequenos.

POSTGRESQL

```
const { rows } =
  await pool.query(
  `SELECT * FROM alunos
   WHERE id = $1`, [id]);
const aluno = rows[0];
```

Sobrevive. Feito para muitos usuários.

O resto da rota — o 404 e o res.json(aluno) — é **igual nas três**. Muda só onde os dados moram.

PRÁTICA 5 • DESAFIO • 50 MIN • DUPLAS

A API do desafio, agora no banco

Peguem a API que a dupla fez no desafio da aula passada e troquem a lista em memória pelo **PostgreSQL**.

Cinema
`filmes`
titulo, genero, duracao

Biblioteca
`livros`
titulo, autor, disponivel

Academia
`clientes`
nome, plano, ativo

Pet shop
`pets`
nome, especie, tutor

Oficina
`ordens`
placa, servico, status

Tarefas
`tarefas`
descricao, concluida

ENTREGAS

1. Tabela no pgAdmin, com id SERIAL e os campos do cenário.
2. As cinco rotas com pool.query e $1, $2…
3. Status 201, 204, 400, 404 — e 500 com try/catch.
+ **Desafio:** filtro com WHERE, como /filmes?genero=comedia.

PENSE E RESPONDA Que tipo de coluna vocês escolheram para cada campo? Por quê?

04 • FECHAMENTO

Erros comuns e como resolver

Sintoma	Causa provável	Solução
ENOENT: no such file or directory	Arquivo não existe, ou o node rodou de outra pasta	Monte o caminho com path.join e existsSync
Unexpected end of JSON input	O arquivo .json está vazio	Escreva [] dentro dele
A rota responde {}	Faltou o await	await na chamada e async na rota
A alteração some no próximo GET	Mudou a lista, mas não salvou	Chame o salvar antes de responder
Servidor reinicia a cada POST	O nodemon vigia arquivos .json	Use node --watch
password authentication failed	Usuário ou senha errados no db.js	Confira os dados de conexão
relation "alunos" does not exist	Tabela não criada, ou banco errado no db.js	Confira o database e a tabela
invalid input syntax for type integer	O id da rota não é número: /alunos/dois	Valide o id e responda 400
Preço volta como "6.50"	O pg devolve NUMERIC como texto	Converta com Number() se precisar

04 • FECHAMENTO

Confira antes de chamar o professor

ARQUIVO JSON

- [ ] Caminho com path.join; arquivo criado se faltar.
- [ ] Ler e salvar com await, em rotas async.
- [ ] Toda rota que altera: ler, mudar, salvar.

POSTGRESQL

- [ ] Tabela com id SERIAL criada no banco certo.
- [ ] db.js conectado: o servidor sobe sem erro.
- [ ] Todo valor como $1, $2 — nada de juntar texto.

TESTES

- [ ] Dados continuam depois de reiniciar o servidor.
- [ ] Status 200, 201, 204, 400, 404 e 500 conferidos.
- [ ] Mudanças aparecem no pgAdmin.

BACK-END · CONSULTA RÁPIDA

Resumo

Persistir
fora do processo
arquivo ou banco
sobrevive ao reinício

fs
node:fs/promises
readFile(c, 'utf-8')
writeFile(c, texto)

JSON
parse: texto → objeto
stringify: o inverso
(dados, null, 2)

async / await
await espera
quem espera é async
sem await: {}

Arquivo
ler · mudar · salvar
gerarId: maior + 1
comece com []

pg
npm install pg
new Pool({ ... })
export const pool

Consulta
pool.query(sql, [v])
{ rows, rowCount }
um item: rows[0]

Parâmetros
$1, $2 na ordem
nunca juntar texto
evita injeção

Criar
id SERIAL
INSERT sem o id
RETURNING *

Atualizar
COALESCE($1, nome)
o ?? do SQL
sem linhas: 404

Remover
DELETE … $1
rowCount 0: 404
senão: 204

Erros
try / catch
500 em JSON
console.error

BACK-END • AULA CONCLUÍDA

Seus dados sobrevivem!

Termine o CRUD da lanchonete no banco e o desafio em dupla.

PARA ESTUDAR

nodejs.org/api/fs.html — documentação do módulo fs

node-postgres.com — documentação da biblioteca pg

postgresql.org/docs — documentação do PostgreSQL

expressjs.com/pt-br — documentação do Express